

TOWARDS REVERSIBLE MODEL MERGING FOR LOW-RANK WEIGHTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Model merging seeks to consolidate multiple fine-tuned models into a single set of weights that performs well across all source tasks. Prior work has established that such merges can approximate the per-task performance of individually fine-tuned models, yet it has largely overlooked an increasingly prevalent regime: the models to be merged are themselves stored in low-rank form, obtained either through low-rank adaptation (LoRA) or through post-training singular value decomposition (SVD). We show that applying conventional merging methods directly to low-rank weights precipitates severe degradation of the merged model, revealing a structural incompatibility between existing merging heuristics and already-compressed representations. This diagnosis motivates a deliberate reformulation of the objective. Rather than collapsing all adapters into a single task-agnostic set of weights, we retain a compact shared basis—equivalent in storage to as few as two models—together with per-task coefficients, from which any original task-specific model is recovered on demand by linear combination once its task identity is known. RMM is therefore best understood not as a producer of a single merged model but as a task-conditioned reconstruction, or multi-adapter compression, scheme: it trades the assumption of task-agnostic inference for the ability to “revert” to any individual model, in recognition that no single merged model can consistently match one specialized for its task. We introduce Reversible Model Merging (RMM), an efficient, data-free method whose reconstruction objective admits a closed-form solution—a truncated SVD—that jointly selects the optimal shared basis and the associated task-specific coefficients. Across diverse datasets, tasks, model architectures, and compression ranks, RMM preserves substantially more of the per-task performance of low-rank compressed models than single-model merges at matched compression, at a modest and tunable increase in offline storage, and its storage cost grows sublinearly with the number of tasks.

1 INTRODUCTION

Parameter-efficient fine-tuning, most prominently Low-Rank Adapters (LoRA) (Hu et al., 2021), has become a cornerstone of adapting large language models to diverse downstream tasks without incurring the full cost of retraining. In parallel, post-training compression—notably singular value decomposition (SVD) truncation of weight updates—is now routinely applied to curtail storage and inference costs, furnishing a complementary axis of efficiency (Ryu et al., 2023; Ping et al., 2024; Wang et al., 2025b;a). Against this backdrop, model merging has emerged as a compelling paradigm for consolidating the knowledge of multiple task-specific models into a single unified set of weights. A rapidly maturing body of merging techniques—spanning simple parameter interpolation through optimization-based procedures—has established that a merged model can perform competitively across its source tasks without access to the original training data (Matena & Raffel, 2022; Yang et al., 2024; Yadav et al., 2023; Wei et al., 2025). These two trajectories, however, have advanced in isolation: the merging literature has been developed and validated almost exclusively on full-rank models, leaving the interaction between merging and low-rank compression essentially unexamined.

This gap in the literature matters because a decisive limitation persists even in the full-rank setting: the most sophisticated merging strategies still fall short of the performance of the original individually fine-tuned models. Our experiments show that this shortfall does not merely persist but widens dramatically once the models being merged have already been compressed into low-rank

form, whether through LoRA fine-tuning or post-training SVD truncation. The cause is structural rather than incidental. Because a single merged model must serve every task at once, merging necessarily collapses distinct task-specific representations into one shared parameter space, and this collapse induces cross-task interference that erodes the specialized adaptations each model had acquired (Gargiulo et al., 2025). Low-rank compression amplifies precisely this failure mode: it removes the representational headroom on which full-rank merging implicitly relies to absorb such interference, leaving the compressed subspaces too constrained to accommodate competing tasks simultaneously.

Although an individual low-rank adapter or SVD-truncated update is inherently compact, the proliferation of such artifacts introduces a new scalability bottleneck at the level of the collection. Practical multi-task, federated, and continual-learning pipelines routinely accumulate dozens or even hundreds of task-specific low-rank adapters, each incurring separate storage and coordination overhead; retaining all of them erodes the very storage savings that low-rank parameterization was designed to deliver and renders deployment progressively more cumbersome. This tension motivates methods that operate directly on low-rank representations at the collection level—not merely to consolidate storage and suppress redundancy, but to enable efficient sharing, deployment, and transfer of compressed models across a growing roster of tasks.

To characterize these limitations concretely, we begin by analyzing how existing merging techniques behave when applied to low-rank compressed models. The analysis centers on the deltas—the weight differences between each fine-tuned model and the shared pre-trained initialization—which conventional strategies combine into a single set of parameters under an implicit assumption of linear compatibility across tasks. We show empirically that merging low-rank deltas in this manner precipitates a catastrophic loss of task performance. Two compounding factors account for the failure: the constrained expressive capacity intrinsic to low-rank representations, and the misalignment of task-specific subspaces which, having each been optimized independently, interfere destructively when superimposed. Together these factors mark a genuine incompatibility rather than a tuning artifact, and they set the agenda for the reformulation that follows.

This observation prompts us to reconsider the objective itself, and to ask: *What if the goal were not to produce a single final model? What if we relaxed the problem to permit retaining more than one model? What if we preserved the ability to reconstruct the original models on demand?* Rather than collapsing multiple low-rank adapters into one set of weights, we maintain a compact shared basis together with per-task coefficients, from which each original model can be recovered through linear combination. We are explicit that this is a deliberate change of operating point rather than a drop-in replacement for task-agnostic merging: because the coefficients are task-specific, reconstruction requires the identity of the target task at inference time, which we assume is supplied by the user or by a routing mechanism. Under that assumption, the reformulation opens an underexplored design space in which, instead of compressing all information into a single model, we retain complementary representational directions across a small set of basis models and thereby enable “reversion” to any original adapter. We therefore recast the problem as the construction of a reconstruction-capable model space—equivalently, as task-conditioned multi-adapter compression—and evaluate it against single-model merging under matched compression, where task-agnostic collapse is precisely the behavior we seek to avoid.

Building on this reconstruction-driven perspective, we develop a theoretical characterization of the basis that minimizes reconstruction error and derive from it our method, Reversible Model Merging (RMM). RMM casts the problem as basis selection: identify a compact set of directions that supports accurate reconstruction of all original low-rank factors, and determine the per-task coefficients whose linear combination recovers each individual adaptation. We summarize our contributions as follows. **(i) Diagnosis.** We identify and analyze the failure of conventional merging under low-rank compression, attributing it to two compounding mechanisms—the limited representational capacity of compressed subspaces and the destructive misalignment of independently optimized task subspaces—and document its severity across two compression methods and four ranks. **(ii) Reformulation and closed-form solution.** We reframe the problem as the construction of a reconstruction-capable model space, formalize it as an orthonormal basis-selection objective over the low-rank factor vectors, and show that its optimum is given exactly, without iterative optimization, by a truncated SVD of the stacked task vectors. **(iii) Method.** We instantiate this framework as RMM, an efficient and data-free procedure that requires no retraining and exposes a single interpretable hyperparameter, the number of retained components p , which directly controls the fidelity–storage trade-off.

(iv) *Empirical evidence.* Across diverse datasets, tasks, model architectures, and compression ranks, RMM preserves far more of the per-task performance of low-rank compressed models than single-model merges at matched compression, and its storage cost grows sublinearly with the number of tasks—properties that make it well suited to large-scale multi-task adaptation and allow storage to be traded against reconstruction fidelity on demand.

2 RELATED WORK

Model merging has become a powerful strategy for consolidating knowledge from multiple fine-tuned models. Existing methods fall into two broad families: data-free manipulations of the parameter space, and optimization-based approaches that leverage data or auxiliary objectives. We review representative techniques from both lines below and situate our contribution against them. Two distinctions organize this comparison. First, every method surveyed here ultimately commits to a single task-agnostic merged model, whereas RMM retains per-task coefficients and reconstructs a task-specific adapter on demand; the two therefore occupy different operating points, and RMM should be read as the multi-adapter-compression counterpart to task-agnostic merging rather than as a strictly superior instance of it. Second, these methods were developed and validated on full-rank models, whereas RMM is designed for, and evaluated in, the low-rank regime that they do not address.

Task Arithmetic: The foundational idea in model merging is task arithmetic (Ilharco et al., 2023), which defines a delta as the difference between the parameters of a fine-tuned model and its pre-trained initialization. Its central insight is that such deltas behave as modular building blocks: they can be added to a base model to impart new abilities, subtracted to suppress unwanted behaviors, or scaled and linearly combined to blend multiple capabilities. This framework established a durable foundation for composing model behaviors directly in weight space, and it furnishes the delta representation on which the methods below—and our own—operate.

DARE: While simple delta addition can be effective, it frequently degrades when the merged models carry conflicting or overlapping parameter updates. To mitigate this, Yu et al. (2024) introduced DARE (Drop and Rescale), a technique grounded in aggressive randomized pruning. DARE discards the vast majority of parameter changes—up to 90–99%—on the premise that most updates are inessential and may actively interfere with knowledge contributed by other models; the surviving updates are then rescaled to preserve the overall magnitude of the modification, retaining core task-specific adaptations while attenuating interference.

TIES: In contrast to DARE’s randomized pruning, TIES-merging adopts a deterministic procedure for resolving parameter conflicts (Yadav et al., 2023), proceeding in three phases—Trim, Elect Sign, and Merge—that address inter-model interference systematically. It first trims all but the largest-magnitude parameters, so that only the most influential fine-tuning updates survive; it then examines, at each parameter position, the signs of the updates across models and enforces agreement by discarding contributions that oppose the elected majority sign; finally, it averages the sign-consistent parameters into the merged representation. This consensus-driven procedure curbs destructive interference and yields a more stable merged model. Sharing this interference-reduction motivation, EMR-Merging (Elect, Mask, and Rescale) elects a unified model and attaches task-specific binary masks and rescalers derived from each individual model, steering the unified model toward the target task at inference time (Huang et al., 2024)—an early instance of retaining lightweight per-task information alongside a shared model, though still organized around a single elected set of weights.

Other Merging Methods: Beyond purely parameter-space manipulations, a second family of merging approaches relies on optimization procedures or on direct access to a small dataset. Model soups (Wortsman et al., 2022) refine the interpolation weights of task arithmetic by minimizing validation loss, steering the merge toward better generalization. Fisher-weighted averaging (Matena & Raffel, 2022) instead incorporates second-order information, weighting each parameter by its estimated importance for task performance. A further line of work casts merging explicitly as an optimization problem: RegMean (Jin et al., 2023) and AdaMerging (Yang et al., 2024) search for task-dependent merging coefficients by solving small-scale objectives defined, respectively, on the discrepancy between the predictions of the merged and individual models and on the entropy of predictions over unlabeled data. More recently, Wei et al. (2025) formulate merging as minimizing the loss difference between the merged and individual fine-tuned models, employing a first-order

Taylor expansion to obtain a tractable surrogate. Although these methods can attain stronger task-specific performance, they generally require labeled or unlabeled data, additional optimization, or both, which constrains their applicability precisely in the setting we target, where consolidation must proceed without retraining and without data. RMM operates squarely in this data-free regime and, in contrast to all of the above, retains the capacity to reconstruct each source model rather than committing to one merged solution—the property that distinguishes multi-adapter compression from task-agnostic merging, and that we later show is essential to preserving per-task performance under low-rank compression.

3 PRELIMINARIES

We first formalize the conventional problem of consolidating a collection of fine-tuned models into a single unified model, which fixes the notation and the baseline objective against which RMM is later contrasted. Let $\{\theta_1, \dots, \theta_n\}$ denote n models fine-tuned on n distinct tasks. A merging algorithm $\mathcal{M}$ maps these models to a single set of weights $\theta_{\mathcal{M}} = \mathcal{M}(\{\theta_i\}_{i=1}^n)$, and the objective is to identify a procedure $\mathcal{M}$ that maximizes the average validation performance across all tasks:

$$\max_{\mathcal{M}} \frac{1}{n} \sum_{i=1}^n \text{Perf}(\theta_{\mathcal{M}}, \mathcal{D}_i), \quad (1)$$

where $\mathcal{D}_i$ denotes a held-out validation set for task i and Perf denotes any suitable performance metric (e.g., accuracy or F1 score). Following standard practice, we assume that all fine-tuned models originate from a common pretrained initialization θ_{pre} . For any layer l , the update, or delta, of model i relative to the base model is $\Delta_i^l = \theta_i^l - \theta_{\text{pre}}^l$, with $\Delta_i^l \in \mathbb{R}^{m \times d}$ the weight difference for a layer of dimensions $m \times d$. Merging operates on these deltas, and the low-rank structure they acquire under compression is the object of the next subsection.

3.1 LOW-RANK COMPRESSED MODELS

In practice, fine-tuned models are routinely compressed into low-rank form to reduce storage, training, and inference costs. We consider the two most widely adopted compression strategies, which differ in whether the low-rank structure is imposed during or after fine-tuning:

LoRA fine-tuning: During training, each layer is augmented with trainable low-rank factors $A \in \mathbb{R}^{m \times r}$ and $B \in \mathbb{R}^{r \times d}$, where $r \ll \min(m, d)$. The pretrained weights remain frozen, and only A and B are optimized, so the update is low-rank by construction.

Post-training SVD truncation (PT-SVD): After standard full-rank fine-tuning, the update Δ_i^l is decomposed via SVD and truncated to its top- r singular components, yielding the low-rank approximation $\hat{\Delta}_i^l = A_i^l B_i^l$ with $A_i^l \in \mathbb{R}^{m \times r}$ and $B_i^l \in \mathbb{R}^{r \times d}$, where $r \ll \min(m, d)$.

In both cases, the fine-tuned update for layer l of model i is expressed in the common low-rank factorized form $\hat{\Delta}_i^l = A_i^l B_i^l$, with $A_i^l \in \mathbb{R}^{m \times r}$, $B_i^l \in \mathbb{R}^{r \times d}$, and r the compression rank. The consolidation problem thus reduces to combining low-rank updates across tasks: the goal is to construct a merged model that integrates the compressed deltas $\hat{\Delta}_i = \{\hat{\Delta}_i^l\}_{l=1}^L$ over all tasks,

$$\hat{\theta}_{\mathcal{M}} = \theta_{\text{pre}} + \mathcal{M}(\{\hat{\Delta}_i\}_{i=1}^n), \quad \max_{\mathcal{M}} \frac{1}{n} \sum_{i=1}^n \text{Perf}(\hat{\theta}_{\mathcal{M}}, \mathcal{D}_i), \quad (2)$$

where L denotes the total number of layers. This low-rank setting, in which every model is stored in compressed form, is the focus of our work throughout. The next section investigates how conventional merging strategies behave under it, and shows that the reduction is far from benign.

4 MOTIVATING OBSERVATION

We now examine how conventional merging strategies behave when applied to the low-rank compressed models introduced in the previous section. For each layer l , prior work has considered two natural extensions of merging to the factorized setting (Tang et al., 2025; Zhao et al., 2025; Zheng et al., 2025; Zhang et al., 2025):

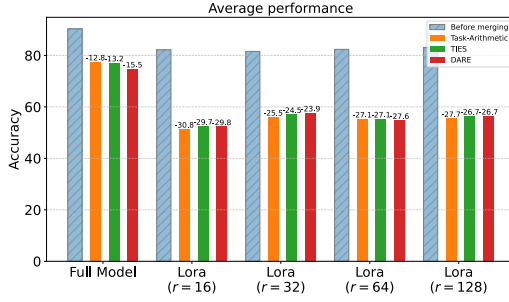


Figure 1: Average task performance of individual models before merging (the upper-bound reference) versus the merged model produced by each merging method, for fully fine-tuned models and for models fine-tuned with LoRA at ranks $r \in \{16, 32, 64, 128\}$. Whereas full-rank merging stays close to the reference, merging the LoRA-compressed models collapses average performance far below it, and the gap persists undiminished across all four ranks.

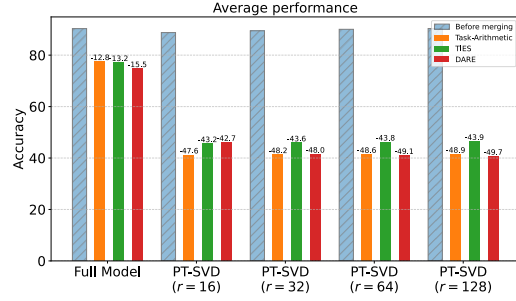


Figure 2: Average task performance of individual models before merging versus the merged model produced by each merging method, for fully fine-tuned models and for models compressed with PT-SVD at ranks $r \in \{16, 32, 64, 128\}$. The catastrophic degradation seen under LoRA recurs here, indicating that the failure is intrinsic to merging low-rank representations rather than an artifact of a particular compression method.

- **Combined merging:**

$$\hat{\theta}_{\mathcal{M}}^l = \theta_{\text{pre}}^l + \mathcal{M}(\{A_i^l B_i^l\}_{i=1}^n),$$

where each low-rank update is first reconstructed into its full-rank form and then merged.

- **Separate merging:**

$$\hat{\theta}_{\mathcal{M}}^l = \theta_{\text{pre}}^l + \mathcal{M}(\{A_i^l\}_{i=1}^n) \mathcal{M}(\{B_i^l\}_{i=1}^n),$$

where the low-rank factors are merged independently before recomposition.

Although both approaches are formally valid, the combined strategy is prohibitively inefficient: storing the merged update in combined form requires materializing a dense $m \times d$ matrix, whereas the separate approach stores only low-rank factors of the same shape as $A_i^l \in \mathbb{R}^{m \times r}$ and $B_i^l \in \mathbb{R}^{r \times d}$. Since $r(m + d) \ll md$ whenever $r \ll \min(m, d)$, the memory footprint of the combined approach negates the very purpose of compression. As a concrete illustration, take a layer with $m = d = 1024$ and compression rank $r = 16$: the combined approach must store a full update of $md = 1,048,576$ parameters, whereas the separate approach stores only $r(m + d) = 16 \times (1024 + 1024) = 32,768$.

This is a **32 $\times$ reduction in storage**, and it establishes that the combined approach entirely forfeits the efficiency benefits of low-rank compression. We therefore adopt the separate strategy as the only practically viable route for merging compressed models and treat it as the setting for all subsequent analysis.

Under the separate approach, we applied several established merging baselines to low-rank compressed models across a range of compression ranks, using RoBERTa-base (Zhuang et al., 2021) as the pre-trained model and evaluating on five GLUE tasks (Wang et al., 2018) (full protocol and per-task numbers in Appendix A.3). As Figures 1 and 2 make plain, merging the compressed models induces a *catastrophic drop* in average task performance relative to merging their full-rank counterparts, and this collapse holds uniformly across both compression methods and all four ranks. We attribute the degradation to two compounding mechanisms: (i) **amplified task interference**—compression discards much of the representational capacity, leaving low-rank models markedly more susceptible to destructive interference when their updates are superimposed; and (ii) **loss of coupling between A and B**—the separate strategy merges the factors A_i^l and B_i^l independently, disregarding the coupling inherent in their product $A_i^l B_i^l$ and thereby injecting structural misalignment into the merged updates. Acting jointly, these mechanisms severely impair the performance of merged low-rank models and expose a deeper incompatibility between conventional merging and compressed representations.

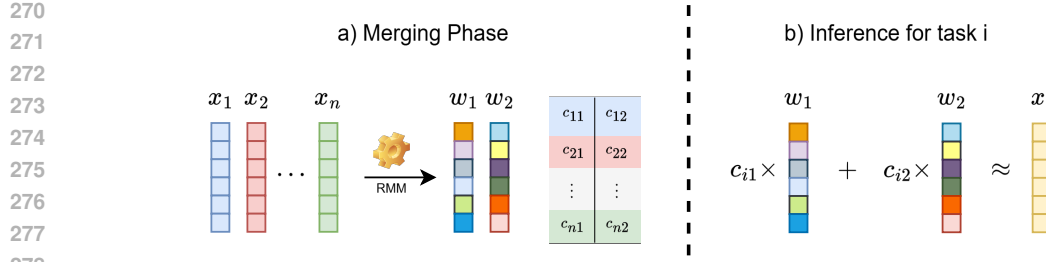


Figure 3: Overview of the proposed RMM framework. **Left:** in the merging phase, RMM computes, for each task-vector position, a compact orthonormal basis (two directions, in this example) together with the associated task-specific coefficients. **Right:** at inference, once the target task i is known, its adapter is reconstructed by linearly combining the shared basis vectors with model i 's coefficients and restoring the mean. Because the basis is shared across all tasks, admitting an additional task costs only one further coefficient vector—the source of RMM's sublinear storage growth.

This evidence crystallizes a central insight: under conventional strategies, no single task-agnostic merged model can reliably recover the task-specific performance of the original low-rank adapters. The natural question follows: *can we instead reconstruct the original compressed models, or closely approximate them, from a compact set of basis models—equivalent in storage to as few as two adapters and scalable to the available budget?* Rather than forcing all updates into a single representation, we pursue an alternative design space in which consolidation constructs a *reconstruction-capable model space* and defers the choice of task to inference time. We formalize this perspective and introduce our method, *Reversible Model Merging* (RMM), in the next section.

5 REVERSIBLE MODEL MERGING

The empirical failures documented above expose a fundamental limitation: no single task-agnostic merged model can faithfully preserve the task-specific performance of all low-rank compressed models at once. RMM responds by relinquishing that goal deliberately. Instead of collapsing all models into one set of weights, it constructs a small set of shared components—the basis models—from which every original task model can be reconstructed to high fidelity, given the task identity at inference time. The advantage over merging is thus obtained not by resolving interference within a single model but by declining to force one, and we characterize RMM accordingly as a task-conditioned reconstruction scheme rather than as a task-agnostic merge.

Rather than forcing all n task-specific models into one merged solution $\hat{\theta}_{\mathcal{M}}$, RMM retains a compact set of p shared components with $2 \leq p \ll n$. These components span a low-dimensional subspace that captures the dominant variation across tasks, enabling efficient on-demand recovery of each model. The formulation exposes an explicit performance–memory trade-off governed by a single interpretable hyperparameter p : increasing p enlarges the shared subspace, improving reconstruction fidelity at a proportional increase in storage, so practitioners can dial the operating point to the budget at hand.

For each layer l , we operate on the r -dimensional vectors that constitute the low-rank delta factors A_i^l and B_i^l —specifically, the rows of A_i^l and the columns of B_i^l , where r is the compression rank. We treat each such row and each such column as an independent *task vector* and refer to a fixed row or column index as a *task-vector position*. For any fixed position, we collect the corresponding vectors across all n models and denote the one contributed by model i as $x_i \in \mathbb{R}^r$. RMM is then applied independently at each position: the set $\{x_i\}_{i=1}^n$ is compressed by projection onto a position-specific shared basis, from which its members can later be reconstructed on demand. Each layer contributes $m + d$ task-vector positions in total— m rows from A_i^l and d columns from B_i^l —and the procedure is carried out separately for each, so that the shared basis is local to a position rather than global across the factor matrices.

To this end, we seek a set of basis vectors $\{w_1, \dots, w_p\}$ with each $w_j \in \mathbb{R}^r$ for $1 \leq j \leq p$, collected as $\mathbf{W} = [w_1 \dots w_p] \in \mathbb{R}^{r \times p}$, whose span is the subspace onto which each task vector x_i is projected so as to reconstruct it as accurately as possible. The coefficient matrix $\mathbf{C} \in \mathbb{R}^{n \times p}$

carries, in its i -th row $[c_{i1}, \dots, c_{ip}]$, the scalar weights that linearly combine the basis vectors to reconstruct $\mathbf{x}_i$. Determining $\mathbf{W}$ and $\mathbf{C}$ jointly so as to minimize the total squared reconstruction error over all n task vectors yields the optimization problem

$$\min_{\mathbf{W}, \mathbf{C}} \sum_{i=1}^n \left\| \mathbf{x}_i - \sum_{j=1}^p c_{ij} \mathbf{w}_j \right\|_2^2. \quad (3)$$

This formulation stores rp parameters for the basis and np for the coefficients, for a total of $p(r+n)$ —a substantial reduction from the rn parameters needed to retain all task vectors directly whenever $p \ll \min(r, n)$. The hyperparameter p governs the performance–storage trade-off, and Figure 3 illustrates the construction for $p = 2$. The objective in equation 3 admits the equivalent matrix form

$$\min_{\mathbf{W}, \mathbf{C}} \left\| \mathbf{X} - \mathbf{C}\mathbf{W}^\top \right\|_F^2, \quad (4)$$

where $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]^\top \in \mathbb{R}^{n \times r}$ stacks all task vectors row-wise. For any fixed $\mathbf{W}$, the optimal coefficient matrix is the least-squares solution $\mathbf{C}^* = \mathbf{X}\mathbf{W}(\mathbf{W}^\top \mathbf{W})^{-1}$, available in closed form. Imposing orthonormality on the basis ($\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p$ with $p < r$) makes the projection an orthogonal one and reduces the problem to

$$\min_{\mathbf{W}} \left\| \mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top \right\|_F^2 \quad \text{s.t.} \quad \mathbf{W}^\top \mathbf{W} = \mathbf{I}_p. \quad (5)$$

In this formulation we store $\mathbf{W} \in \mathbb{R}^{r \times p}$ and $\mathbf{X}\mathbf{W} \in \mathbb{R}^{n \times p}$, where $\mathbf{X}\mathbf{W}$ is exactly the coefficient matrix $\mathbf{C}$ of equation 4.

Theorem 1 (Optimal Solution to equation 5). *Let $\mathbf{X} \in \mathbb{R}^{n \times r}$ be zero-centered and $p < \min\{r, n\}$. The optimal basis $\mathbf{W}^*$ that minimizes equation 5 is given by the top- p eigenvectors of the sample covariance matrix $\mathbf{X}^\top \mathbf{X}$. Equivalently, $\mathbf{W}^*$ consists of the top- p right singular vectors of $\mathbf{X}$.*

The proof of Theorem 1 is given in Appendix A.1; it establishes that the optimal reconstruction basis is obtained in closed form, without any iterative optimization. The two phases of RMM—an offline merging phase that builds the representation and an inference phase that reconstructs a chosen adapter—follow directly.

5.1 MERGING PHASE

The merging phase constructs a compact representation of all task-specific models around a shared basis, and executes entirely offline, prior to any inference. For each layer and each task-vector position (a fixed row of $\mathbf{A}_i^l$ or column of $\mathbf{B}_i^l$), we assemble the task vectors of all n models into $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]^\top \in \mathbb{R}^{n \times r}$. We first compute the mean vector $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ and zero-center the data, $\mathbf{X} \leftarrow [\mathbf{x}_1 - \boldsymbol{\mu} \dots \mathbf{x}_n - \boldsymbol{\mu}]^\top$, as Theorem 1 requires. We then take the SVD of the centered matrix, $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, and set the basis to its top- p right singular vectors, $\mathbf{W}^* = \mathbf{V}_{[:,1:p]} \in \mathbb{R}^{r \times p}$, which by the theorem minimize the reconstruction error in equation 5. Finally, the coefficient matrix follows as $\mathbf{C}^* = \mathbf{X}\mathbf{W}^* \in \mathbb{R}^{n \times p}$, whose i -th row $[c_{i1}, \dots, c_{ip}]$ encodes exactly the weights that recombine the basis into $\mathbf{x}_i$.

Once this phase completes, the original task vectors $\{\mathbf{x}_i\}_{i=1}^n$ are discarded, and only the basis $\mathbf{W}^*$, the coefficients $\mathbf{C}^*$, and the mean vector $\boldsymbol{\mu}$ are retained per position. This reduces the per-position storage from the rn parameters needed to keep all task vectors to $p(r+n) + r$, where the trailing r accounts for the retained mean; this figure is exactly the storage cost carried into the experimental comparisons.

5.2 INFERENCE PHASE

Reconstruction is task-conditioned: it presupposes the identity of the target task, which we assume is either specified explicitly by the user or resolved by a router that assigns each input to the most suitable model index. Our experiments isolate the reconstruction quality of RMM by supplying this identity through an oracle, so the reported numbers should be read as the performance attainable given correct routing; the accuracy of an automatic router is orthogonal to the compression mechanism and is not evaluated here. Once the target index i is known, the task vector $\mathbf{x}_i$ is

reconstructed—as $\hat{x}_i$ —from the compact representation (W^*, C^*, μ) stored for each task-vector position during the merging phase, and the factors A_i^l and B_i^l are recovered by concatenating these reconstructions across their respective row and column positions.

Recall that, for each task-vector position, $W^* \in \mathbb{R}^{r \times p}$ holds the orthonormal basis, $C^* \in \mathbb{R}^{n \times p}$ the task-specific coefficients, and $\mu \in \mathbb{R}^r$ the mean over all task vectors. Reconstruction linearly combines the basis directions with model i ’s coefficients and restores the mean shift μ :

$$\hat{x}_i = \sum_{j=1}^p c_{ij}^* w_j^* + \mu = C_{[i,:]}^* W^{*\top} + \mu.$$

The resulting $\hat{x}_i \in \mathbb{R}^r$ approximates the original task vector x_i discarded during merging. Performed independently for every task-vector position (each row of A_i^l and each column of B_i^l) and across all layers, this reconstruction recovers the complete low-rank delta $\hat{\Delta}_i$ of task-specific model i . Algorithm 1 sets out the full merging and inference procedure; we now turn to the empirical evaluation of RMM against standard merging baselines.

Algorithm 1 Reversible Model Merging (RMM)

```

1: Merging Phase:
2: Require: Low-rank deltas  $A_i^l$  and  $B_i^l$ , compression rank  $r$ , number of basis vectors  $p$ 
3: for  $l = 1$  to  $L$  do
4:   for each task vector position (each row in  $A_i^l$  or each column in  $B_i^l$ ) do
5:     Gather task vectors  $x_i$  of each model  $i$  ( $1 \leq i \leq n$ )
6:      $X \leftarrow [x_1 \dots x_n]^\top \in \mathbb{R}^{n \times r}$ 
7:      $\mu \leftarrow \sum_{i=1}^n x_i \in \mathbb{R}^r$ 
8:      $X \leftarrow [x_1 - \mu \dots x_n - \mu]^\top$ 
9:      $U, \Sigma, V^\top \leftarrow \text{SVD}(X)$ 
10:     $W^* \leftarrow V_{[:,1:p]} \in \mathbb{R}^{r \times p}$ 
11:     $C^* \leftarrow X W^* \in \mathbb{R}^{n \times p}$ 
12:    Store  $(W^*, C^*, \mu)$  and discard task vectors  $\{x_1 \dots x_n\}$ 
13:   end for
14: end for
15: Inference Phase:
16: Require: target task  $i$ , stored representation  $(W^*, C^*, \mu)$  for each task vector position
17: for  $l = 1$  to  $L$  do
18:   for each task vector position do
19:      $\hat{x}_i = C_{[i,:]}^* W^{*\top} + \mu$ 
20:   end for
21:    $\hat{A}_i^l = \text{Concatenate}(\hat{x}_i \text{ vectors across all row positions})$ 
22:    $\hat{B}_i^l = \text{Concatenate}(\hat{x}_i \text{ vectors across all column positions})$ 
23: end for
24:  $\hat{\Delta}_i = \{\hat{A}_i^l \hat{B}_i^l\}_{l=1}^L$ 
25: Inference with the reconstructed model  $\theta_{pre} + \hat{\Delta}_i$ 

```

6 EXPERIMENTS

We evaluate RMM across a range of compression settings, tasks, and model architectures. The evaluation is organized around two questions: whether RMM preserves the per-task performance of low-rank task-specific models while reducing the overall storage of the collection, and how the resulting fidelity-storage operating point compares with that of conventional single-model merging baselines under matched compression.

6.1 EXPERIMENTAL SETUP

The primary evaluation uses eight RoBERTa-base models, each fine-tuned on a distinct GLUE task (Zhuang et al., 2021) and then individually compressed with either PT-SVD or LoRA at target ranks

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	RTE	COLA	STS-B	Average
16	No merging	100%	92.02	85.68	93.69	85.21	87.19	77.98	58.05	90.59	83.80
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.19	31.66
	TIES	13%	50.56	34.78	49.08	31.82	36.82	47.29	0.00	3.75	31.76
	DARE	13%	50.80	33.51	49.08	31.82	36.82	47.29	0.00	5.07	31.80
	RMM ($p = 2$)	50%	51.20	78.20	88.30	66.74	86.02	47.29	6.56	33.59	57.24
	RMM ($p = 3$)	69%	88.83	86.26	92.89	79.58	87.18	48.74	33.85	60.46	72.22
32	No merging	100%	92.31	85.86	94.15	86.38	88.62	78.34	59.33	90.68	84.46
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.21	31.66
	TIES	13%	50.56	35.19	49.08	31.82	36.82	47.29	0.00	3.87	31.83
	DARE	13%	50.72	33.51	49.08	31.82	36.82	47.29	0.00	3.48	31.59
	RMM ($p = 2$)	44%	50.54	74.32	55.05	62.68	87.07	47.29	0.00	23.18	50.02
	RMM ($p = 3$)	59%	84.66	86.32	92.89	79.70	88.05	47.29	23.57	44.54	68.38
64	No merging	100%	92.51	86.03	94.38	87.18	90.15	78.70	61.11	90.72	85.10
	TA	13%	50.61	33.51	49.08	31.82	36.82	47.29	0.00	4.23	31.67
	TIES	13%	50.58	35.13	49.08	31.82	36.82	47.29	0.00	3.80	31.82
	DARE	13%	50.58	33.51	49.08	31.82	36.82	47.29	0.00	5.22	31.79
	RMM ($p = 2$)	41%	50.53	73.10	49.08	58.48	86.98	47.29	0.00	17.50	47.87
	RMM ($p = 3$)	55%	74.14	86.20	92.43	78.83	89.00	47.29	13.16	33.70	64.34
128	No merging	100%	92.68	86.14	94.50	87.33	91.05	79.78	60.16	90.74	85.30
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.23	31.67
	TIES	13%	50.58	35.54	49.08	31.82	36.82	47.29	0.00	3.93	31.88
	DARE	13%	50.61	34.78	49.08	31.82	36.82	47.29	0.00	2.41	31.60
	RMM ($p = 2$)	39%	50.54	70.78	49.08	54.69	84.68	47.29	0.00	14.53	46.45
	RMM ($p = 3$)	52%	59.20	85.80	92.32	77.50	89.02	47.29	6.56	26.94	60.58

Table 1: Consolidating eight PT-SVD-compressed RoBERTa-base models on the GLUE benchmark at ranks $r = 16, 32, 64, 128$. Per-task metrics follow the GLUE protocol and **Average** is their mean; **Storage** is the fraction of parameters retained relative to keeping all eight adapters, inclusive of basis, coefficients, and means. *No merging* is the individual-model upper bound. At every rank both RMM variants outperform the single-model baselines TA, TIES, and DARE by a wide margin—at a correspondingly larger storage budget—and RMM ($p = 3$) recovers the largest share of the upper bound.

$r \in \{16, 32, 64, 128\}$. We apply RMM to the resulting low-rank models and compare it against three standard merging baselines—Task Arithmetic (TA) (Ilharco et al., 2023), TIES-merging (Yadav et al., 2023), and DARE (Yu et al., 2024)—all operated on the same compressed factors under the separate strategy of Section 4. The performance of the individual compressed models prior to any merging serves as the natural upper bound, since no consolidation of a compressed collection can exceed the compressed models it is built from. To probe generality across architecture, scale, modality, and task count, Appendix A.2 reports further experiments on the decoder-only OPT-1.3b (Zhang et al., 2022) and on ViT-B/32 (Radford et al., 2021) for vision classification.

Following the GLUE protocol (Wang et al., 2018), each task is scored with its standard metric: QNLI (question-answering inference), MRPC (paraphrase detection), SST-2 (sentiment classification), MNLI (multi-genre natural language inference), QQP (duplicate-question detection), and RTE (textual entailment) by accuracy; CoLA (linguistic acceptability) by Matthews correlation; and STS-B (semantic textual similarity) by Pearson correlation. Overall performance is summarized by the mean score across tasks. For every method we also report the relative storage cost, defined as the fraction of parameters retained relative to storing all individual task-specific adapters, and computed so as to include all quantities required for reconstruction. A single merged model incurs a fixed cost of $\frac{r}{rn} = \frac{1}{n}$, whereas RMM with p basis components incurs $\frac{p(r+n)+r}{rn}$ —the numerator accounting for the shared basis, the per-task coefficients, and the retained mean per position, exactly as derived in Section 5.1. Reading performance jointly with this cost is central to the comparison, because the two methods occupy different operating points: the single-model baselines minimize storage at the expense of per-task fidelity, while RMM spends a modest, tunable increment of offline storage to preserve that fidelity.

6.2 RESULTS AND DISCUSSION

Tables 1 and 2 report RMM against the baselines under PT-SVD and LoRA compression, respectively. Across both compression methods and all four ranks, RMM preserves substantially more of the average task performance than TA, TIES, and DARE, and the margin is large and uniform. At

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	RTE	COLA	STS-B	Average
16	No merging	100%	61.38	83.48	94.50	83.90	87.56	68.23	53.92	88.71	77.71
	TA	13%	50.59	33.51	72.13	34.79	64.46	47.29	0.00	21.81	40.57
	TIES	13%	50.63	33.68	74.31	35.64	67.45	47.29	0.00	26.38	41.92
	DARE	13%	50.63	33.51	73.74	35.97	65.35	47.29	0.00	21.62	41.01
	RMM ($p = 2$)	50%	51.13	34.14	90.71	60.48	84.41	47.29	0.00	43.14	51.41
	RMM ($p = 3$)	69%	55.92	38.14	93.12	82.95	87.30	47.29	6.56	57.81	58.64
32	No merging	100%	60.13	81.28	93.81	84.46	88.18	62.82	53.12	88.53	76.54
	TA	13%	50.91	68.06	54.70	33.77	65.50	55.96	0.00	-0.48	41.05
	TIES	13%	51.03	68.06	56.77	34.24	66.27	58.12	0.00	1.02	41.94
	DARE	13%	50.96	64.64	63.07	35.12	63.66	55.60	0.00	1.01	41.76
	RMM ($p = 2$)	44%	52.08	65.39	81.54	62.46	77.06	57.40	0.00	9.95	50.73
	RMM ($p = 3$)	59%	54.62	69.97	92.20	81.29	85.97	57.40	4.64	24.08	58.77
64	No merging	100%	59.62	84.12	94.15	85.20	88.75	67.15	58.31	88.70	78.25
	TA	13%	54.75	68.52	77.29	37.24	36.82	47.29	0.00	-3.53	39.80
	TIES	13%	54.82	68.75	80.73	37.67	36.82	47.29	0.00	-2.48	40.45
	DARE	13%	53.71	68.58	78.56	38.33	36.82	47.29	0.00	-4.28	39.88
	RMM ($p = 2$)	41%	55.74	69.28	87.27	42.81	77.72	47.29	0.00	4.91	48.13
	RMM ($p = 3$)	55%	59.07	70.14	90.83	79.11	82.06	47.29	0.00	14.70	55.40
128	No merging	100%	61.58	85.04	93.58	85.71	89.68	57.04	59.35	89.68	77.71
	TA	13%	51.49	69.22	54.47	37.16	63.27	47.29	0.00	-1.74	40.15
	TIES	13%	51.91	68.93	57.34	37.81	63.36	47.29	0.00	-0.25	40.80
	DARE	13%	51.97	68.99	58.60	35.64	63.33	47.29	0.00	-0.44	40.67
	RMM ($p = 2$)	39%	57.18	67.83	88.53	49.54	84.64	47.29	0.00	28.00	52.88
	RMM ($p = 3$)	52%	59.13	39.19	92.09	80.08	86.39	47.29	15.45	71.18	61.35

Table 2: Consolidating eight LoRA-compressed RoBERTa-base models on the GLUE benchmark at ranks $r = 16, 32, 64, 128$. Columns follow the convention of Table 1. The advantage of RMM over TA, TIES, and DARE observed under PT-SVD carries over to LoRA compression, indicating that the gains are not tied to a single compression method.

the lower ranks ($r = 16, 32$) the single-model baselines fail catastrophically, averaging only 31–42% and collapsing almost entirely on structurally sensitive tasks such as CoLA and STS-B; the interference documented in Section 4 is fully in evidence here. RMM already recovers a large share of the lost performance at $p = 2$ (for instance 57.24% under PT-SVD and 51.41% under LoRA at $r = 16$), and $p = 3$ improves further still (to 72.22% and 58.64% at $r = 16$), sharply narrowing the gap to the individual compressed models. At the higher ranks ($r = 64, 128$), where compression error is smaller, the ordering persists: with $p = 3$, RMM attains 64.34% and 60.58% under PT-SVD and 55.40% and 61.35% under LoRA, while the single-model baselines remain near 40%. Even $p = 2$ retains a clear advantage throughout. These gains, however, are not obtained under matched storage—RMM’s advantage reflects the additional per-task information it retains (39–69% of the full collection across the two variants, versus the baselines’ fixed 13%), and is therefore the expected consequence of operating at a higher-storage, task-conditioned point rather than evidence of a superior task-agnostic merge. Read in that light, the tables establish the intended claim precisely: by declining to collapse the adapters into one model, the reversible formulation sidesteps the destructive interference that cripples single-model merging on low-rank representations.

We further observe that the relative gain of RMM over the baselines contracts as the rank grows. A plausible explanation is that higher-rank task representations occupy higher-dimensional, more expressive, and less mutually aligned subspaces, which are intrinsically harder to summarize with a shared low-dimensional basis; on this reading the effect is a property of the representations rather than a limitation of the method, and it is consistent with the fidelity–storage trade-off that p controls. Taken together, the results establish that RMM offers a consistent and favorable operating point across both compression regimes and all ranks: it preserves high per-task performance at the cost of a modest, tunable increase in storage, with the number of reversible components serving as an explicit accuracy–storage dial. The additional cost is moreover borne by **offline memory**—typically CPU-resident and inexpensive—rather than by the runtime footprint that governs inference quality, which makes the trade especially attractive when preserved accuracy is the binding objective.

6.3 SCALABILITY OF RMM

We next examine how RMM scales as the number of models n grows. Figures 4 and 5 plot the storage ratio, relative to storing all individual task-specific models, against the number of tasks for $p = 2$

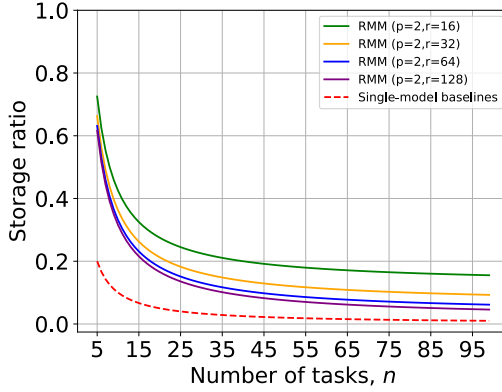


Figure 4: Scalability of RMM ($p = 2$): the storage ratio—memory retained relative to storing all models—as a function of the number of tasks. The ratio declines steadily as tasks accumulate, so that absolute storage grows sublinearly in the number of tasks.

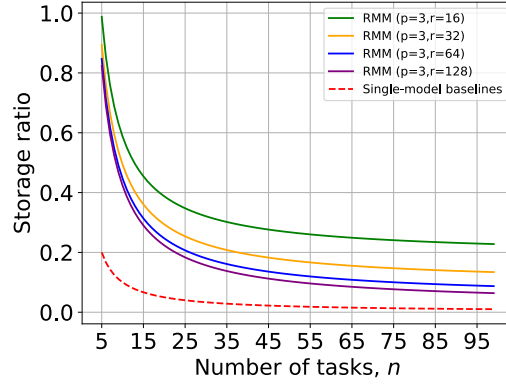


Figure 5: Scalability of RMM ($p = 3$): the storage ratio—memory retained relative to storing all models—as a function of the number of tasks. As at $p = 2$, the ratio decreases with the task count, so the higher-fidelity setting inherits the same favorable sublinear scaling.

and $p = 3$. Both settings scale favorably: the relative cost falls steadily as tasks accumulate, so that absolute storage grows *sublinearly* in the number of task-specific models—the defining efficiency property of the reversible formulation. The sublinearity follows directly from the design: because the shared basis is amortized across all tasks, admitting a new task appends only a single row to the coefficient matrix rather than an entire model. This is precisely the regime in which RMM’s higher per-collection storage relative to a single merged model is repaid, since the incremental cost per task shrinks as the roster lengthens. Conventional approaches face the opposite choice—collapse to one merged model and sacrifice per-task performance, or retain every model independently and pay linearly in n —which positions RMM as an attractive intermediate for large-scale multi-task, continual, and federated deployments, where many models must be managed efficiently without forfeiting task performance.

7 CONCLUSION

We introduced Reversible Model Merging, a principled framework that reframes how a collection of low-rank fine-tuned models is stored and recovered. Rather than forcing all task-specific knowledge into a single task-agnostic model, RMM retains a compact shared basis—substantially smaller than the collection, yet strictly more expressive than a single aggregate—together with per-task coefficients, from which each original model is accurately reconstructed once its task identity is known. The reconstruction objective admits a closed-form optimum via truncated SVD, and its single hyperparameter p exposes a controllable fidelity–storage trade-off. Across two compression methods, four ranks, three architectures, and both language and vision tasks, this design preserves far more of the per-task performance of compressed models than single-model merging, at a modest and tunable increase in offline storage, and with a storage cost that grows sublinearly in the number of tasks. We are explicit that these gains arise from a change of operating point: RMM does not resolve the interference internal to a single merged model but avoids incurring it, trading task-agnostic inference for the ability to revert to any specialized adapter.

Two boundaries frame the contribution and mark the natural next steps. First, reconstruction is task-conditioned and our evaluation supplies the task identity through an oracle, so integrating and stress-testing an automatic router—and quantifying RMM’s sensitivity to routing error—is the principal open question for deployment. Second, RMM occupies a higher-storage, higher-fidelity point than a single merged model, which invites systematic comparison against other multi-adapter compression strategies at matched budgets. More broadly, by casting the consolidation of low-rank models as a structured, reversible process with a closed-form solution, the framework enlarges the

design space of efficient multi-task deployment and points toward performance-aware compression and reconstruction as a fertile direction for future work.

REFERENCES

- Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3606–3613, 2014.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300. URL <https://aclanthology.org/N19-1300/>.
- Antonio Andrea Gargiulo, Donato Crisostomi, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, and Emanuele Rodola. Task singular vectors: Reducing task interference in model merging. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 18695–18705, 2025.
- Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019.
- Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2 edition, 2012.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. Emr-merging: Tuning-free high-performance model merging. *Advances in Neural Information Processing Systems*, 37:122741–122769, 2024.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=6t0Kwf8-jrj>.
- Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=FCnohuR6AnM>.
- Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *2013 IEEE International Conference on Computer Vision Workshops*, pp. 554–561, 2013. doi: 10.1109/ICCVW.2013.77.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Michael S Matena and Colin A Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022.
- Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011. URL http://ufldl.stanford.edu/housenumbers/nips2011_housenumbers.pdf.

- Bowen Ping, Shuo Wang, Hanqing Wang, Xu Han, Yuzhuang Xu, Yukun Yan, Yun Chen, Baobao Chang, Zhiyuan Liu, and Maosong Sun. Delta-come: Training-free delta-compression with mixed-precision for large language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 31056–31077. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/37664246a1e07e212ddacea6e5a523f2-Paper-Conference.pdf.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 8748–8763. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/radford21a.html>.
- Simo Ryu, Seunghyun Seo, and Jaejun Yoo. Efficient storage of fine-tuned models via low-rank approximation of weight residuals. *arXiv preprint arXiv:2305.18425*, 2023.
- Johannes Stalkamp, Marc Schlippsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: A multi-class classification competition. In *The 2011 International Joint Conference on Neural Networks*, pp. 1453–1460, 2011. doi: 10.1109/IJCNN.2011.6033395.
- Dennis Tang, Prateek Yadav, Yi-Lin Sung, Jaehong Yoon, and Mohit Bansal. LoRA merging with SVD: Understanding interference and preserving performance. In *ICML 2025 Workshop on Reliable and Responsible Foundation Models*, 2025. URL <https://openreview.net/forum?id=t9FrMviTaP>.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In Tal Linzen, Grzegorz Chrupała, and Afra Alishahi (eds.), *Proceedings of the 2018 EMNLP Workshop Black-boxNLP: Analyzing and Interpreting Neural Networks for NLP*, pp. 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5446. URL <https://aclanthology.org/W18-5446/>.
- Xin Wang, Samiul Alam, Zhongwei Wan, Hui Shen, and Mi Zhang. SVD-LLM v2: Optimizing singular value truncation for large language model compression. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4287–4296, Albuquerque, New Mexico, April 2025a. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.217. URL <https://aclanthology.org/2025.naacl-long.217/>.
- Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. SVD-LLM: Truncation-aware singular value decomposition for large language model compression. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=LNyIUouhdt>.
- Yongxian Wei, Anke Tang, Li Shen, Zixuan Hu, Chun Yuan, and Xiaochun Cao. Modeling multi-task model merging as adaptive projective gradient descent. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=EoKRSR5Pa>.
- Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pp. 23965–23998. PMLR, 2022.
- Jianxiong Xiao, James Hays, Krista A. Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 3485–3492, 2010. doi: 10.1109/CVPR.2010.5539970.

- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=xtaX3WyCj1>.
- Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. Adamerging: Adaptive model merging for multi-task learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=nZP6NgD3QY>.
- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*, 2024.
- Juzheng Zhang, Jiacheng You, Ashwinee Panda, and Tom Goldstein. LoRI: Reducing cross-task interference in multi-task low-rank adaptation. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=b8cW86QcOD>.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- Ziyu Zhao, Tao Shen, Didi Zhu, Zexi Li, Jing Su, Xuwu Wang, and Fei Wu. Merging loRAs like playing LEGO: Pushing the modularity of loRA to extremes through rank-wise clustering. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=j6fsbpAl1N>.
- Shenghe Zheng, Hongzhi Wang, Chenyu Huang, Xiaohui Wang, Tao Chen, Jiayuan Fan, Shuyue Hu, and Peng Ye. Decouple and orthogonalize: A data-free framework for lora merging. *arXiv preprint arXiv:2505.15875*, 2025.
- Liu Zhuang, Lin Wayne, Shi Ya, and Zhao Jun. A robustly optimized BERT pre-training approach with post-training. In Sheng Li, Maosong Sun, Yang Liu, Hua Wu, Kang Liu, Wanxiang Che, Shizhu He, and Gaoqi Rao (eds.), *Proceedings of the 20th Chinese National Conference on Computational Linguistics*, pp. 1218–1227, Huhhot, China, August 2021. Chinese Information Processing Society of China. URL <https://aclanthology.org/2021.ccl-1.108/>.

A APPENDIX

A.1 PROOF OF THEOREM 1

Let $\mathbf{A} = \mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top$. By definition of the Frobenius norm, $\|\mathbf{A}\|_F^2 = \text{Tr}(\mathbf{A}^\top \mathbf{A})$, where $\text{Tr}(\cdot)$ denotes trace of a square matrix. Expanding gives:

$$\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 = \text{Tr}((\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)).$$

We expand the term in $\text{Tr}(\cdot)$:

$$\begin{aligned} (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top) &= (\mathbf{X}^\top - \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top)(\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top) \\ &= \mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top - \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X} \\ &\quad + \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top. \end{aligned}$$

Next, we take the trace of the expanded form. Using cyclicity of trace and $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p$:

$$\begin{aligned} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - 2\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) + \text{Tr}(\mathbf{W}^\top \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) \\ &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - 2\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) + \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) \\ &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}). \end{aligned}$$

Since $\text{Tr}(\mathbf{X}^\top \mathbf{X})$ is a constant, minimizing $\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2$ is equivalent to maximizing $\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W})$. Hence,

$$\min_{\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 \iff \max_{\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p} \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}).$$

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	BOOLQ	Average
16	No merging	100%	91.74	71.54	95.30	84.19	88.14	71.47	83.73
	Task-Arithmetic	17%	50.34	37.16	58.49	34.43	63.30	41.87	47.60
	TIES	17%	49.79	36.75	61.24	35.57	63.55	40.67	47.93
	DARE	17%	49.95	37.68	56.08	34.51	63.39	42.78	47.40
	RMM ($p = 2$)	63%	88.12	44.00	93.58	73.06	85.63	59.69	74.01
	RMM ($p = 3$)	85%	91.58	49.04	95.64	80.31	87.52	65.50	78.27
32	No merging	100%	92.06	72.17	95.41	84.60	89.17	72.45	84.31
	Task-Arithmetic	17%	50.28	37.22	58.94	34.57	63.33	41.93	47.71
	TIES	17%	49.77	36.93	61.93	35.69	63.65	40.73	48.12
	DARE	17%	50.39	40.87	56.19	34.96	63.59	43.33	48.22
	RMM ($p = 2$)	56%	86.66	41.91	91.74	71.75	85.68	56.39	72.36
	RMM ($p = 3$)	76%	91.85	44.81	94.84	80.21	88.43	63.52	77.28
64	No merging	100%	92.26	72.12	95.41	84.64	90.07	72.84	84.56
	Task-Arithmetic	17%	50.28	37.28	59.06	34.56	63.34	42.08	47.77
	TIES	17%	49.75	37.04	61.81	35.84	63.74	40.76	48.16
	DARE	17%	50.16	40.52	58.83	35.59	63.35	42.54	48.50
	RMM ($p = 2$)	53%	83.87	42.72	89.45	69.76	85.70	53.43	70.82
	RMM ($p = 3$)	71%	92.07	41.97	94.50	79.70	88.73	61.96	76.49
128	No merging	100%	92.35	72.41	95.41	84.67	90.93	73.09	84.81
	Task-Arithmetic	17%	50.32	37.39	58.94	34.63	63.37	42.08	47.79
	TIES	17%	49.79	37.04	62.04	35.99	63.74	40.55	48.19
	DARE	17%	50.03	38.14	57.91	34.72	63.31	42.11	47.70
	RMM ($p = 2$)	52%	81.38	42.78	84.75	67.66	83.22	51.99	68.63
	RMM ($p = 3$)	69%	92.28	41.80	93.69	79.42	88.76	59.69	75.94

Table 3: Consolidating six PT-SVD-compressed OPT-1.3b models on GLUE-derived tasks at ranks $r = 16, 32, 64, 128$. Columns follow the convention of Table 1. On this larger decoder-only architecture, both RMM variants again preserve far more per-task performance than TA, TIES, and DARE at every rank, and RMM ($p = 3$) approaches the individual-model upper bound.

By the Rayleigh quotient theorem (Horn & Johnson, 2012), this maximum is attained by taking the columns of W to be the top- p eigenvectors of $X^\top X$. This is exactly the principal component analysis (PCA) solution: it identifies the p -dimensional subspace that maximizes the projected variance and, equivalently, minimizes the squared reconstruction error. Writing the SVD of X as $X = U\Sigma V^\top$ gives $X^\top X = V\Sigma^2 V^\top$, so the eigenvectors of $X^\top X$ coincide with the right singular vectors of X , the columns of V ; the same optimal basis is therefore obtained directly from the top- p right singular vectors. In practice this SVD-based route is numerically more stable and often computationally preferable to explicitly forming $X^\top X$, particularly when r is large. The result is thus the standard optimality of truncated SVD for squared reconstruction error, which we invoke deliberately: its value here lies less in mathematical novelty than in supplying an exact, iteration-free optimum for the reconstruction objective around which RMM is built.

A.2 ADDITIONAL RESULTS

This section presents extended results on two additional pre-trained backbones, OPT-1.3b and ViT-B/32, that probe RMM’s generality along the axes of architecture, scale, modality, and task count. For **OPT-1.3b**, we evaluate on six natural language understanding tasks drawn from GLUE and related benchmarks: QNLI, MRPC, SST-2, MNLI, QQP, and BoolQ (Clark et al., 2019). For **ViT-B/32**, we evaluate on seven diverse vision classification datasets: MNIST (LeCun et al., 1998), GTSRB (Stallkamp et al., 2011), SVHN (Netzer et al., 2011), Cars (Krause et al., 2013), EuroSAT (Helber et al., 2019), DTD (Cimpoi et al., 2014), and SUN397 (Xiao et al., 2010).

Table 3 reports the six PT-SVD-compressed OPT-1.3b models. Both RMM variants preserve markedly more per-task performance than the single-model baselines at every rank, and RMM ($p = 3$) closes most of the gap to the individual models while retaining substantially fewer parameters than the full collection. Table 4 reports the seven ViT-B/32 models, where the advantage is even more pronounced: at $p = 3$, RMM attains an average accuracy within 3% of the individual models while reducing storage to 59% of that required to retain all models at $r = 128$, and the same pattern holds across the remaining ranks. Together, these results show that RMM reconstructs the original compressed models faithfully across architectures, scales, and modalities at a favorable storage cost, so that the behavior established on RoBERTa/GLUE is a property of the method rather than of a single backbone or task family.

r	Method	Storage	MNIST	GTSRB	SVHN	Cars	EuroSAT	DTD	SUN397	Average
16	No merging	100%	99.61	97.22	97.06	69.20	99.67	75.21	72.72	87.24
	Task-Arithmetic	14%	47.19	28.29	38.64	56.27	57.52	41.38	62.92	47.46
	TIES	14%	60.14	34.52	49.03	58.55	61.70	43.67	62.87	52.93
	DARE	14%	53.80	31.00	44.96	57.60	56.93	43.99	62.42	50.10
	RMM ($p = 2$)	55%	99.16	88.52	94.89	66.61	97.44	60.64	67.86	82.16
	RMM ($p = 3$)	76%	99.58	95.65	96.54	68.30	98.93	67.18	69.50	85.10
32	No merging	100%	99.65	98.45	97.35	73.04	99.78	78.72	75.52	88.93
	Task-Arithmetic	14%	47.90	28.31	38.68	56.24	57.52	41.38	63.00	47.58
	TIES	14%	61.25	35.28	49.95	58.80	62.41	44.04	63.05	53.54
	DARE	14%	53.05	31.04	46.34	58.54	57.74	42.87	62.87	50.35
	RMM ($p = 2$)	49%	98.98	88.23	94.29	68.32	96.81	61.91	68.36	82.41
	RMM ($p = 3$)	67%	99.50	96.53	96.68	70.90	98.93	70.43	71.23	86.31
64	No merging	100%	99.67	98.68	97.41	75.76	99.85	79.15	77.73	89.75
	Task-Arithmetic	14%	47.99	28.30	38.71	56.21	57.59	41.38	63.22	47.63
	TIES	14%	61.83	35.93	50.28	59.08	63.07	44.15	63.17	53.93
	DARE	14%	54.78	32.79	45.05	58.23	61.30	42.87	62.77	51.11
	RMM ($p = 2$)	46%	98.46	84.43	92.83	69.36	95.59	62.39	69.77	81.83
	RMM ($p = 3$)	62%	99.45	96.14	96.33	72.84	98.56	72.07	73.85	87.03
128	No merging	100%	99.68	98.75	97.46	76.99	99.85	79.04	78.59	90.05
	Task-Arithmetic	14%	48.00	28.31	38.73	56.21	57.63	41.44	63.35	47.67
	TIES	14%	62.08	36.36	50.48	59.23	63.41	44.68	63.25	54.21
	DARE	14%	55.16	32.07	45.47	58.55	61.85	43.09	62.85	51.29
	RMM ($p = 2$)	44%	97.22	78.56	90.63	70.22	94.00	61.49	71.03	80.45
	RMM ($p = 3$)	59%	99.28	94.59	95.95	74.24	98.19	71.86	76.02	87.16

Table 4: Consolidating seven PT-SVD-compressed ViT-B/32 models on vision classification tasks at ranks $r = 16, 32, 64, 128$. Columns follow the convention of Table 1. The advantage of RMM extends to the vision modality: RMM ($p = 3$) trails the individual-model upper bound by only a few points at every rank, whereas the single-model baselines lag far behind.

A.3 RESULTS FOR 5 TASKS

The motivating experiments of Section 4 used RoBERTa-base as the pretrained model and evaluated the merging baselines on five GLUE tasks (QNLI, MRPC, SST-2, MNLI, QQP), as summarized by Figures 1 and 2. For completeness, we report the underlying per-task numbers here: Table 5 corresponds to Figure 1 and covers the LoRA fine-tuned models alongside the performance recovered by RMM, and Table 6 corresponds to Figure 2 and covers the PT-SVD compressed models.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	Average
16	No merging	100%	61.38	83.48	94.50	83.90	87.56	82.16
	Task-Arithmetic	20%	50.61	33.51	73.85	34.23	64.58	51.36
	TIES	20%	50.65	33.51	75.57	35.83	66.97	52.51
	DARE	20%	50.61	33.51	79.36	34.70	63.40	52.32
	RMM ($p = 2$)	73%	59.64	68.64	93.00	82.68	86.89	78.17
	RMM ($p = 3$)	73%	59.64	68.64	93.00	82.68	86.89	78.17
32	No merging	100%	60.13	81.28	93.81	84.46	88.18	81.57
	Task-Arithmetic	20%	51.18	67.30	58.83	34.33	68.56	56.04
	TIES	20%	51.36	68.17	59.52	34.76	71.46	57.05
	DARE	20%	50.83	64.87	68.12	35.54	68.93	57.66
	RMM ($p = 2$)	66%	57.72	67.54	92.55	81.80	86.13	77.15
	RMM ($p = 3$)	66%	57.72	67.54	92.55	81.80	86.13	77.15
64	No merging	100%	59.62	84.12	94.15	85.20	88.75	82.37
	Task-Arithmetic	20%	54.74	68.93	80.28	35.70	66.82	55.29
	TIES	20%	55.26	68.93	78.78	36.76	66.82	55.31
	DARE	20%	54.71	68.81	74.54	38.72	66.82	54.72
	RMM ($p = 2$)	63%	59.82	67.48	91.51	83.21	84.41	77.29
	RMM ($p = 3$)	63%	59.82	67.48	91.51	83.21	84.41	77.29
128	No merging	100%	61.58	85.04	93.58	85.71	89.68	83.12
	Task-Arithmetic	20%	52.52	68.70	54.82	37.83	63.46	55.47
	TIES	20%	52.55	69.04	57.80	38.91	63.72	56.40
	DARE	20%	55.04	69.62	54.47	38.92	64.12	56.43
	RMM ($p = 2$)	62%	60.21	34.96	91.86	78.59	86.65	70.45
	RMM ($p = 3$)	62%	60.21	34.96	91.86	78.59	86.65	70.45

Table 5: Consolidating five LoRA-compressed RoBERTa-base models on GLUE at ranks $r = 16, 32, 64, 128$ (the five-task setting underlying Figure 1). Even at $p = 2$, RMM recovers average performance close to the individual-model upper bound, whereas TA, TIES, and DARE collapse to roughly half that level.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	Average
16	No merging	100%	92.02	85.68	93.69	85.21	87.19	88.76
	Task-Arithmetic	20%	53.03	35.19	49.08	31.82	36.82	41.19
	TIES	20%	55.46	54.32	49.08	31.82	37.21	45.58
	DARE	20%	53.69	58.67	49.08	31.82	37.08	46.07
	RMM ($p = 2$)	73%	79.72	85.16	90.83	76.58	86.09	83.68
32	No merging	100%	92.31	85.86	94.15	86.38	88.62	89.46
	Task-Arithmetic	20%	53.23	35.59	49.08	31.82	36.82	41.31
	TIES	20%	55.78	55.19	49.08	31.82	37.62	45.90
	DARE	20%	55.89	33.74	49.20	31.82	36.85	41.50
	RMM ($p = 2$)	66%	63.87	82.78	91.28	77.12	87.05	80.42
64	No merging	100%	92.51	86.03	94.38	87.18	90.15	90.05
	Task-Arithmetic	20%	53.49	35.88	49.08	31.82	36.82	41.42
	TIES	20%	56.14	55.94	49.08	31.82	38.08	46.21
	DARE	20%	53.51	33.68	49.08	31.82	36.82	40.98
	RMM ($p = 2$)	63%	56.12	81.22	91.06	76.74	88.07	78.64
128	No merging	100%	92.68	86.14	94.50	87.33	91.05	90.34
	Task-Arithmetic	20%	53.60	36.06	49.08	31.82	36.82	41.48
	TIES	20%	56.40	56.35	49.08	31.82	38.61	46.45
	DARE	20%	50.52	34.96	49.08	31.82	36.92	40.66
	RMM ($p = 2$)	62%	54.11	79.19	90.60	74.49	87.46	77.17

Table 6: Consolidating five PT-SVD-compressed RoBERTa-base models on GLUE at ranks $r = 16, 32, 64, 128$ (the five-task setting underlying Figure 2). As under LoRA, RMM ($p = 2$) preserves average performance near the individual-model upper bound while the baselines degrade sharply.